Newton School of Technology

02

Lecture 02

Operating System Foundations II & Process Concept

by - Brijesh Shukla

C07: Operating Systems

THE CENTRAL MYSTERY

Newton School of Technology

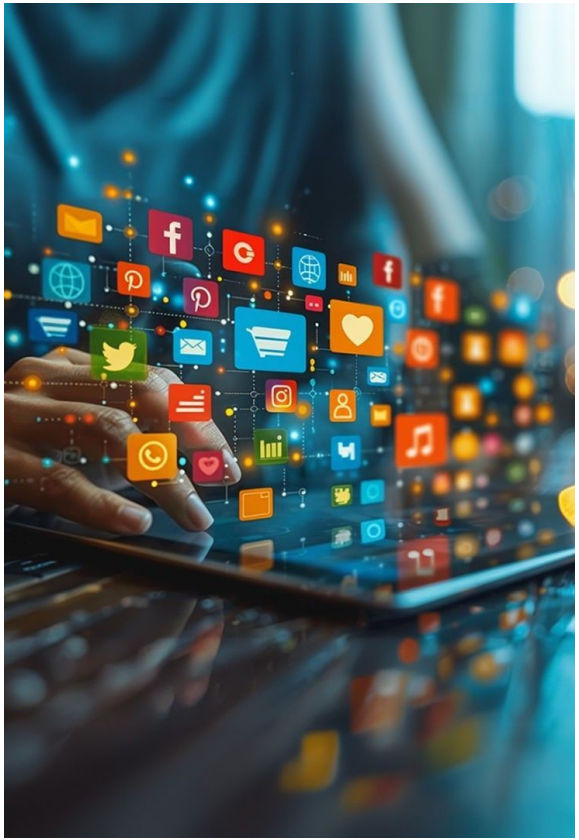
Every Second, Your Mac Performs a Miracle

You double-click an app icon...

💀 **Dead code on disk** → ⚡ **Living process in memory**

But how?

- How does the OS bring code to life?
- How does it manage 100+ processes simultaneously?
- How does it switch between them so fast you don't notice?



THE PROBLEM WE'RE SOLVING

Newton School of Technology

Challenge #1: Design

How do you build an OS that's both FAST and SAFE? → *This shapes everything that follows*

Challenge #2: Multiplicity

How do you run 100 programs on 1 CPU? → *The illusion of parallelism*

Challenge #3: Management

How do you track and control everything happening? → *The OS needs perfect memory*

CHALLENGE #1 → THE DESIGN DILEMMA

Newton School of Technology

The Dilemma: You're building an OS. Where do you put everything?

Option A: All Together (Monolithic)

- Everything in kernel space
- Direct communication = BLAZING FAST ⚡
- But one bug crashes EVERYTHING 💥

Option B: Separated (Microkernel)

- Minimal kernel, services isolated
- Failures contained = SUPER SAFE 🛡️
- But message passing = SLOWER 🐢

The Question: Can we have both?

THE MONOLITHIC WAY

Newton School of Technology

Architecture: One massive kernel with everything inside

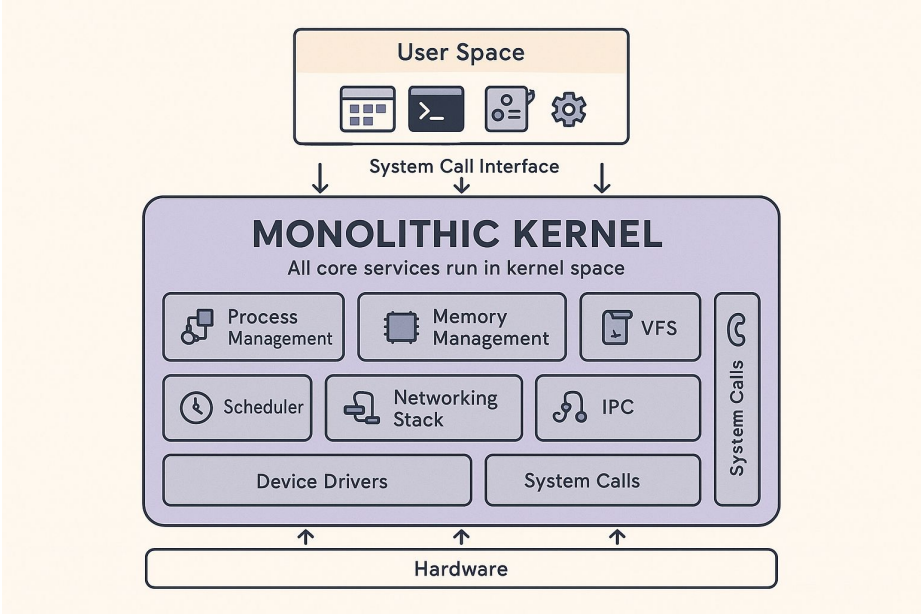
- Device drivers? In kernel.
- File systems? In kernel.
- Networking? In kernel.

✅ **Why It's Awesome:**

- No boundaries = No overhead
- Direct function calls = Lightning speed
- Simple communication between components

❌ **Why It's Terrifying:**

- Graphics driver bug? Kernel panic.
- One mistake = Blue screen of death
- Debugging nightmare



Examples: Linux, Traditional UNIX

Real Talk: This is like having all your house systems in one room. Efficient? Yes. Safe? Not really.

THE MICROKERNEL WAY

Newton School of Technology

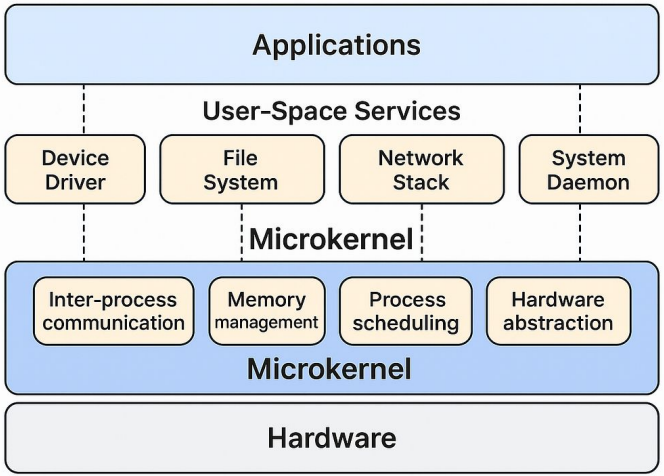
Architecture: Tiny kernel + Everything else in user space

Kernel ONLY does:

- Memory management
- Process scheduling
- Inter-Process Communication (IPC)

Everything else runs as user processes:

- File systems
- Device drivers
- Network stack



✅ **Why It's Beautiful:**

- Driver crashes? Just restart it. System survives. 🛡️
- Easier to debug and maintain
- Modular and flexible

❌ **Why It's Slower:**

- Every operation = Message passing
- Context switches everywhere
- Performance overhead

Examples: QNX (used in cars), MINIX, Mach

THE HYBRID SOLUTION

Newton School of Technology

The Realization: Pure designs are beautiful but impractical

Hybrid Approach:

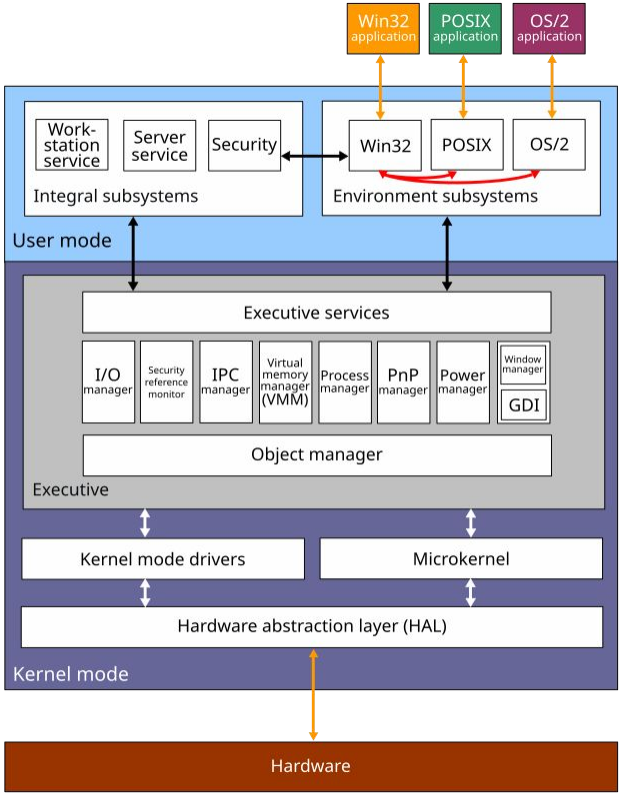
- Critical services in kernel (speed when needed)
- Non-critical services in user space (safety when possible)
- Pragmatism over ideology

🍎 **Your Mac's XNU Kernel = Perfect Example**

Why? Apple wanted Unix compatibility + Mach's elegance + Performance

Other Examples: Windows NT, Android (Linux + userspace framework)

The Lesson: Real-world OS design is about intelligent compromise



ACTIVITY BREAK: ARCHITECTURE

Newton School of Technology

Scenario 1: Desktop OS for creative professionals (video editing, music production)

Scenario 2: Automotive system (Tesla's self-driving software)

Scenario 3: Cloud server running 1000 websites

VIRTUAL MACHINES

Newton School of Technology

The New Problem: What if you want to run Windows AND Linux AND macOS... on ONE machine?

The Solution: Virtual Machines

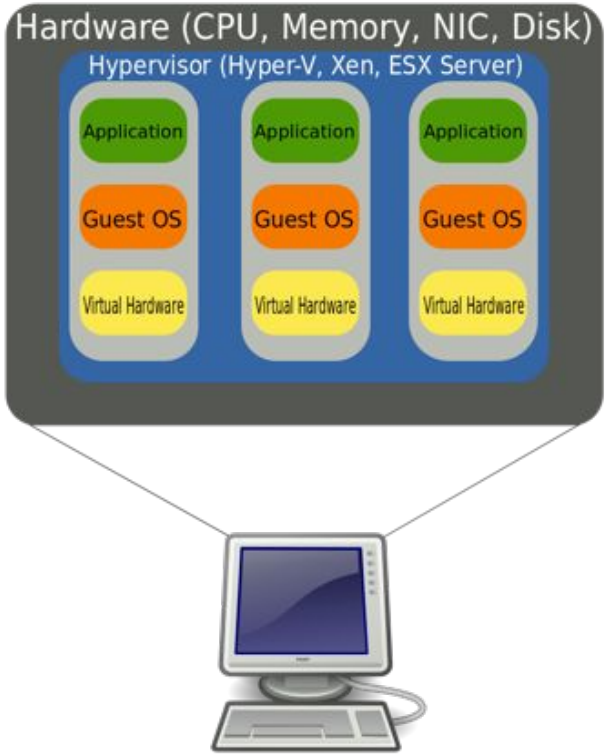
The Magic Trick:

- One physical computer
- Multiple virtual computers
- Each thinks it's alone on real hardware
- All running simultaneously

Who Makes This Possible? The Hypervisor (Virtual Machine Manager)

Why This Matters:

- 🛡️ Isolation (security sandboxes)
- ☁️ Cloud computing foundation (AWS, Azure, Google Cloud)
- 🧪 Testing without risk
- 💰 One server = 50 virtual servers

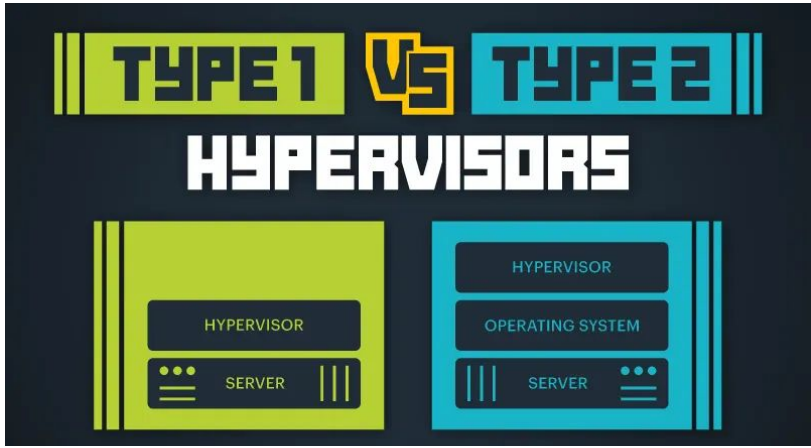


THE TWO TYPES OF HYPERVISORS

Newton School of Technology

Type 1: Bare-Metal Hypervisor

- Runs DIRECTLY on hardware
- Maximum performance
- Enterprise/cloud (VMware ESXi, Xen, KVM, Hyper-V)
- Apple Virtualization Framework (Apple Silicon)



Type 2: Hosted Hypervisor

- Runs ON TOP of host OS
- Easier setup, more overhead
- Development/testing (VMware Fusion, VirtualBox, Parallels)

When to Use:

- Type 1 → Production servers, cloud infrastructure
- Type 2 → Testing on your laptop, running Windows on Mac

Bonus: Containers (Docker) = Even lighter, share kernel

FROM ARCHITECTURE TO EXECUTION

Newton School of Technology

We've designed our OS. We can virtualize. But now...

The Fundamental Question: How does dead code become a living process?

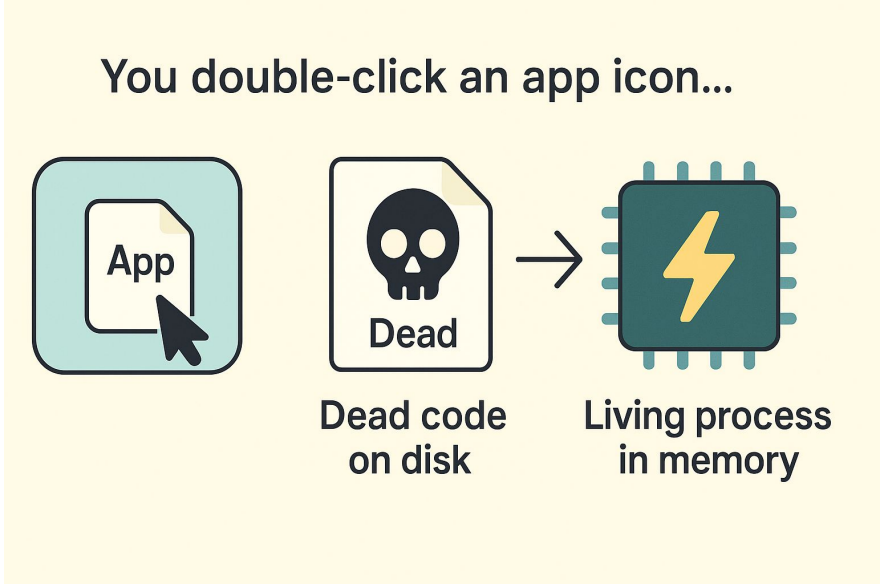
What We Know:

- Your Mac has programs stored on disk (Safari, Chrome, Terminal)
- When you click, something magical happens
- Suddenly it's ALIVE, using CPU, using memory, doing work

What We Don't Know:

- What's the difference between a program and a process?
- How does the OS track 100+ processes?
- How does it switch between them invisibly?

Next: We'll answer these questions and solve Challenge #3



PROGRAM VS PROCESS

Newton School of Technology

📁 **PROGRAM = Recipe**

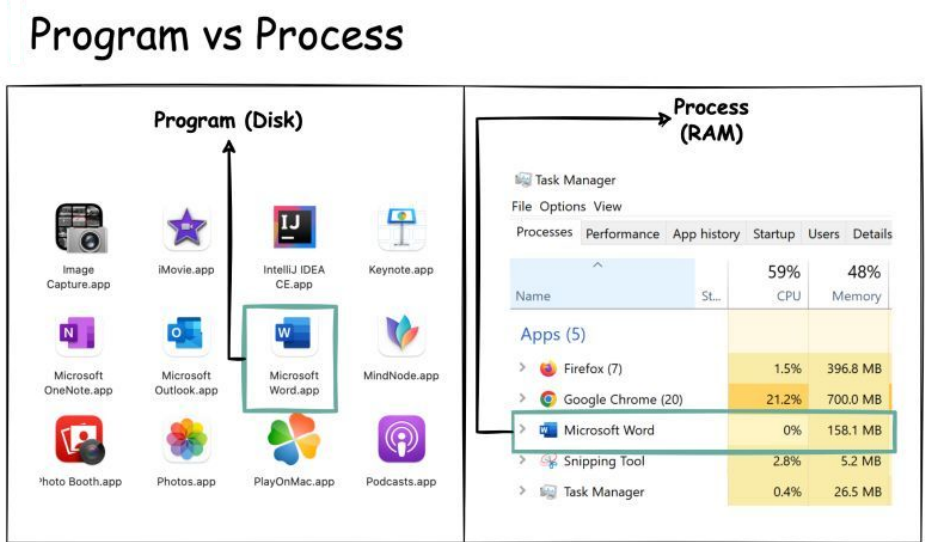
- Static file on disk
- Passive, dormant, lifeless
- Just bytes sitting there
- Example: `/Applications/Safari.app`
- One copy on disk

⚡ **PROCESS = Cooking**

- Program IN EXECUTION
- Active, dynamic, consuming resources
- Has memory, CPU time, state
- Example: Safari.app RUNNING
- Can have multiple processes from one program

The Transformation:

Program (Disk) → OS Loads → Allocates Memory → Creates PCB → Process (RAM)



🖥️ **LIVE DEMO:** Open Activity Monitor. Launch Chrome 3 times.

- 1 Program (Chrome.app)
- 3 Processes (3 separate executions, 3 different PIDs)

PROCESS CONTROL BLOCK (PCB)

Newton School of Technology

The Problem: Your Mac runs 200+ processes. Each needs:

- CPU time allocation
- Memory tracking
- Open files tracking
- Current execution state

The Solution: Process Control Block (PCB)

PCB = A Data Structure for Each Process

Stored Where? Kernel memory (protected, users can't touch it)

The Analogy: PCB is the OS's diary entry about each process

```
struct PCB {
    int    pid;            // Unique ID (like social security #)
    int    state;          // New/Ready/Running/Waiting/Terminated
    long   program_counter; // Next instruction address
    int    registers[32];  // ALL CPU register values
    void*  page_table;     // Memory mapping
    int    priority;       // Scheduling importance
    int    cpu_time_used;  // Total CPU time consumed
    file*  open_files[];   // All open file descriptors
    // ... and more
}
```

THE PROCESS LIFECYCLE → 5 STATES

Newton School of Technology

Every Process Goes Through 5 States:

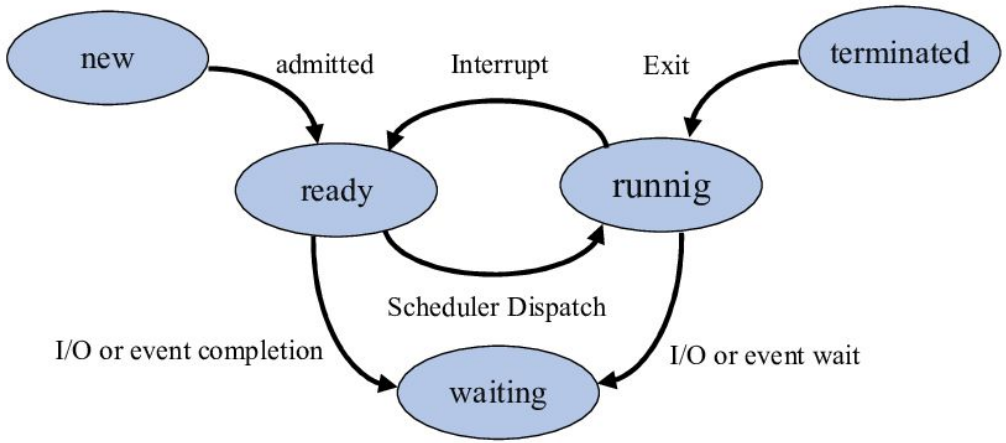
🆕 **NEW** → Being created, OS initializing PCB

⏸️ **READY** → Ready to execute, waiting in queue for CPU

▶️ **RUNNING** → Currently executing on CPU (only 1 per core!)

⏳ **WAITING (BLOCKED)** → Waiting for I/O, event, resource

💀 **TERMINATED** → Finished executing, cleaning up



Key Insights:

- Multiple processes can be READY (ready queue)
- Multiple processes can be WAITING (different events)
- Only ONE process per CPU core can be RUNNING

STATE TRANSITIONS

Newton School of Technology

1. NEW → READY (Admission)

- Process creation complete
- OS admits to ready queue
- PCB fully initialized

2. READY → RUNNING (Dispatch)

- Scheduler selects this process
- Gets CPU time
- Starts executing

3. RUNNING → READY (Preemption)

- Time slice expires (timer interrupt)
- Higher priority process arrives
- Goes back to ready queue

4. RUNNING → WAITING (Block)

- Process requests I/O (disk read, network request)
- Waits for resource
- Calls sleep() or similar

5. WAITING → READY (Wakeup)

- I/O operation completes
- Event occurs
- Resource becomes available
- Back to ready queue (NOT directly to running!)

6. RUNNING → TERMINATED (Exit)

- Process completes execution
- Calls exit() or crashes
- OS reclaims resources

Real Examples:

- You open a file → Running to Waiting
- File opens → Waiting to Ready
- You get CPU → Ready to Running
- Time's up → Running to Ready

CONTEXT SWITCHING

Newton School of Technology

The Situation:

- 200 processes, 8 CPU cores
- Each process thinks it owns the CPU
- OS switches between them thousands of times per second

Context Switch = Saving One Process, Loading Another

Step 1: SAVE Current Process

1. Save ALL register values to PCB
2. Save program counter
3. Save stack pointer
4. Update process state (Running → Ready/Waiting)

Step 2: LOAD Next Process

1. Select next process (scheduler decision)
2. Load its register values from PCB
3. Load its program counter
4. Update state (Ready → Running)
5. Jump to its next instruction

The Cost:

- Time: 1-10 microseconds (pure overhead!)
- Cache pollution (TLB flush, cold cache)
- No useful work during switch

When It Happens:

- ⏰ Timer interrupt (every ~10ms)
- 📞 System call
- 💿 I/O interrupt
- 🚀 Higher priority process ready

The Magic: You never notice because it's so fast

PUTTING IT ALL TOGETHER

Newton School of Technology

The Architecture Foundation:

- macOS uses hybrid kernel (Mach + BSD)
- Balance of speed and safety
- Can run VMs on top for even more isolation

When You Double-Click an App:

Program → Process

1. OS loads program from disk
2. Allocates memory
3. Creates PCB with unique PID
4. State: NEW → READY

Scheduling & Execution

1. Process waits in ready queue
2. Scheduler dispatches: READY → RUNNING
3. Process executes on CPU

Dynamic State Changes

1. Needs I/O? → RUNNING → WAITING
2. I/O done? → WAITING → READY
3. Time up? → RUNNING → READY
4. Finish? → RUNNING → TERMINATED

Context Switching Throughout

1. PCB saves/loads at every transition
2. OS tracks everything
3. You see smooth multitasking

The Miracle: All of this happens millions of times per second, invisibly

THE THREE CHALLENGES → SOLVED

Newton School of Technology

✅ **Challenge #1: Design (Fast AND Safe)**

- **Solution:** Hybrid kernel architecture
- Monolithic where speed matters
- Microkernel where safety matters
- macOS XNU = Real-world example

✅ **Challenge #2: Multiplicity (100 Programs, 1 CPU)**

- **Solution:** Process abstraction + State model
- Each program becomes separate process
- States (Ready/Running/Waiting) manage sharing
- Context switching enables time-sharing

✅ **Challenge #3: Management (Track Everything)**

- **Solution:** Process Control Block (PCB)
- One PCB per process
- Contains ALL information
- Enables state transitions and scheduling

The Big Picture: These aren't separate topics. They're one integrated system solving the fundamental problem of resource management.

REAL-WORLD IMPLICATIONS

Newton School of Technology

Performance Understanding:

- Why closing unused apps helps → Fewer context switches
- Why SSDs feel faster → Less time in WAITING state
- Why RAM matters → More processes fit in memory

Security Understanding:

- Process isolation → One app crash doesn't kill system
- VMs for security → Malware contained in isolated VM
- Kernel/user mode → Protection mechanisms

Development Understanding:

- Why multi-threading matters → More processes/threads
- Why async programming helps → Less blocking (WAITING)
- Why profiling shows states → Optimize state transitions

Cloud Computing:

- VMs = Foundation of AWS/Azure/GCP
- Containers = Even lighter process isolation
- Hypervisors = How cloud providers maximize hardware

Career Relevance:

- Systems programming
- DevOps and cloud architecture
- Performance optimization
- Operating systems development

MIND-BLOWING FACTS

Newton School of Technology

Right Now, On Your Mac:

🔢 **200-300 processes** running simultaneously

⚡ **10,000+ context switches** per second

⏱️ **10ms** typical time slice per process

🧠 **1-10 microseconds** per context switch

💾 **~1-2 KB** per PCB structure

🎯 **Millions** of state transitions per minute

The Realization: Every smooth interaction you have with your Mac is the result of this incredibly complex, precisely orchestrated dance happening thousands of times per second.

You never see any of it. That's the magic of operating systems.

When you move your mouse smoothly across the screen, that's:

- Multiple processes coordinating
- Constant context switches
- PCBs being saved and loaded
- State transitions flowing

COMMON MISCONCEPTIONS

Newton School of Technology

❌ **Myth:** "A program and process are the same thing"

✅ **Reality:** Program is dead code, process is living execution

❌ **Myth:** "Context switching is free"

✅ **Reality:** Pure overhead, 1-10 microseconds of wasted time

❌ **Myth:** "Processes directly switch between Running and Waiting"

✅ **Reality:** Always pass through Ready state

❌ **Myth:** "The OS just 'knows' what each process is doing"

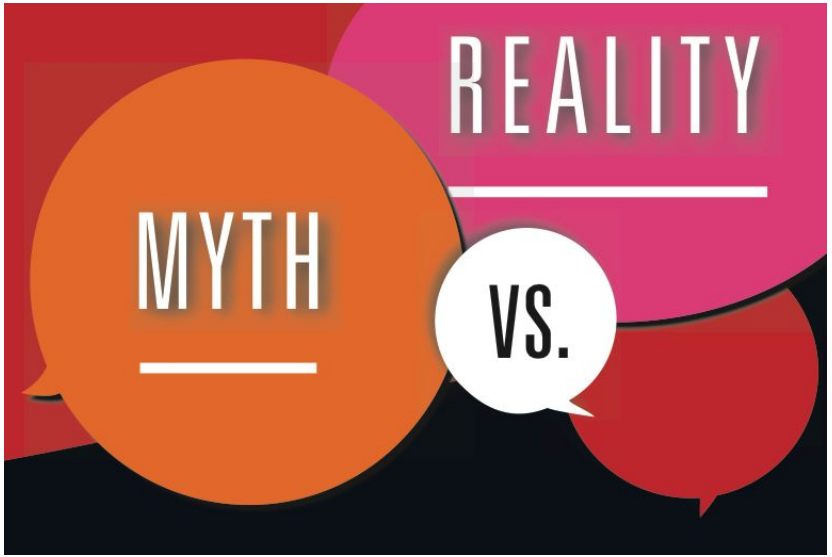
✅ **Reality:** PCB explicitly tracks EVERYTHING

❌ **Myth:** "Virtual machines are slower because they're 'fake'"

✅ **Reality:** Type 1 hypervisors achieve near-native performance

❌ **Myth:** "My Mac runs all programs at the same time"

✅ **Reality:** Time-sharing creates the ILLUSION of parallelism (unless you have as many cores as processes)



DEBUGGING MENTAL MODELS

Newton School of Technology

Scenario 1: You open Safari. It makes a network request. Trace the state transitions.

Answer: NEW → READY → RUNNING → WAITING (network I/O) → READY → RUNNING

Scenario 2: You have 4 CPU cores and 10 processes. How many can be RUNNING simultaneously?

Answer: Maximum 4 (one per core)

Scenario 3: A process's time slice expires. What gets saved?

Answer: Everything in PCB (registers, PC, stack pointer, state)

Scenario 4: Chrome has 3 windows open. How many programs? Processes? PCBs?

Answer: 1 program, 3+ processes (each window + background processes), 3+ PCBs

Scenario 5: Why can't a process go directly from WAITING → RUNNING?

Answer: Must go through READY so scheduler can make fair decisions

THE BIG INSIGHT

Newton School of Technology

Most people think: "I click an icon and the app opens."

The Architecture Decision

- Your Mac's hybrid kernel enables this
- Mach + BSD working together
- Decades of design evolution

The Transformation

- Static program becomes dynamic process
- OS creates entire execution environment
- PCB tracks every detail

The Continuous Dance

- 200+ processes competing for resources
- States transitioning thousands of times per second
- Context switches maintaining the illusion

The Invisible Complexity

- Every smooth interaction = Coordinated chaos
- The OS makes the impossible look effortless

You've pulled back the curtain. You see the machinery behind the magic. You understand your computer at a fundamental level. This is why operating systems is one of the most important courses you'll ever take.

Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.